

FPGA-Based Edge-AI Vision Accelerator

A 3×3 CNN convolution engine at one output pixel per cycle

IEEE SSCS Egypt Chapter • 2026 Student Design Competition

Xilinx Virtex-5 XC5VLX50T • Digilent Genesys • ISE 14.7

WHERE WE LANDED

Headline results

317

Slice LUTs

0

Block RAMs

167.0

MHz F-max

476

mW total power

0.8755

out px / cycle

78 / 78

tests bit-exact

70

cycles to 1st output

2.398×10^{-3}

Figure of Merit

All figures post-place-and-route, all timing constraints met, timing score 0.

What the design had to do

- | | | |
|---|---------------|--|
| 1 | Input | 32×32 grayscale, 8-bit unsigned fixed point |
| 2 | Kernel | N×N with programmable coefficients, 8-bit signed |
| 3 | Stride | 1, with a documented border policy |
| 4 | Output | 16-bit signed minimum, overflow handling explained |
| 5 | Verify | Bit-exact against a Python / MATLAB / C golden model |
| 6 | Report | Utilisation, timing, power, and a Figure of Merit |

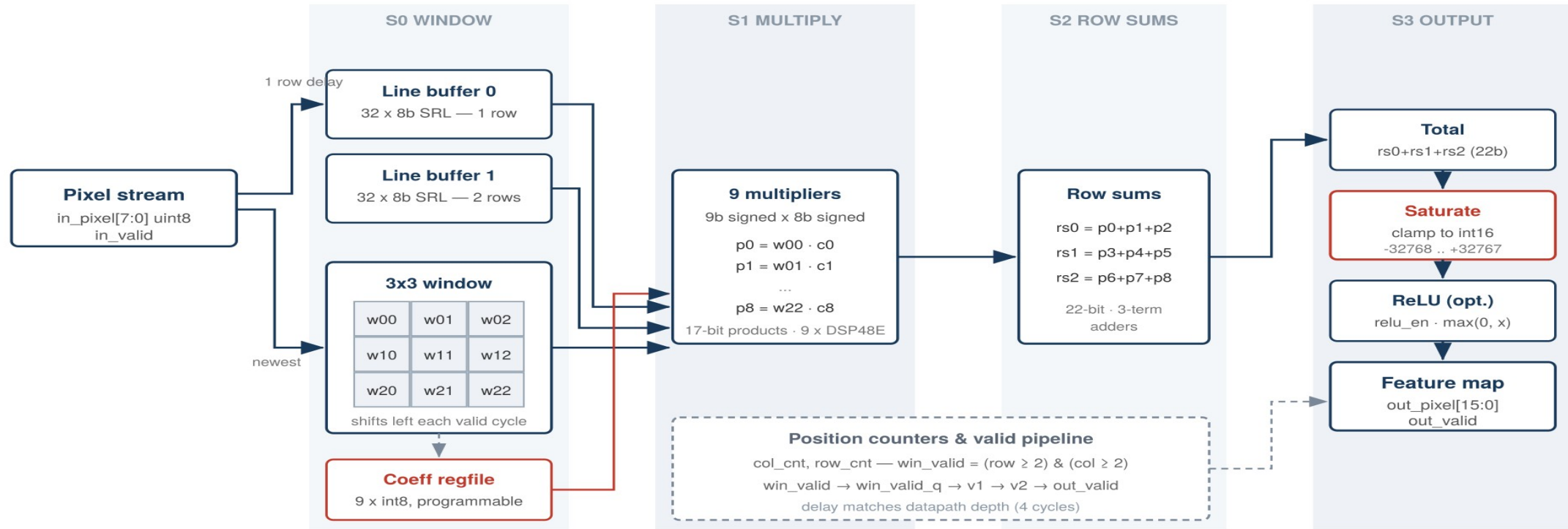
Scored on

- Correctness
- Digital design quality
- Resource usage
- Latency and throughput
- Timing closure
- Power estimate

Architecture

3x3 Convolution Accelerator — Datapath Block Diagram

Xilinx Virtex-5 XC5VLX50T · 1 output pixel/cycle · zero BRAM



Line buffers are flat shift registers, inferred by XST as 16 x 30-bit SRL primitives (17 LUTs) — no block RAM is used.

All state elements are gated on in_valid, so the pipeline is stall-tolerant: gaps in the input stream are harmless.

Resources: 299 LUT · 274 FF · 9 DSP48E · 0 BRAM. Fmax 174.4 MHz (100 MHz target, all constraints met).

One output pixel per cycle

S0**Window**

Two 32-deep SRL line buffers feed a 3×3 register window that shifts left every valid cycle.

S1**Multiply**

Nine parallel 9×8 signed multiplies on DSP48E blocks. 17-bit exact products.

S2**Row sums**

Three 3-term adders. Short combinational depth is why timing closes with 74% headroom.

S3**Output**

Final sum, saturation to int16, then optional ReLU as a single-cycle clamp.

Stall-tolerant by construction

Every state element is gated on `in_valid`, so gaps in the input stream are harmless. The host wrapper therefore advances the stream at UART pace instead of buffering 900 results — one 16-bit register replaces a 14.4 kbit FIFO, which is what keeps BRAM at zero.

Fixed-point analysis

Worst-case accumulator excursion, computed analytically in the golden model:

Kernel	Max positive	Max negative	Bits needed	Saturation
3×3	+291,465	-293,760	20	Required
5×5	+809,625	-816,000	21	Required
7×7	+1,586,865	-1,599,360	22	Required

22-bit accumulator implemented — headroom to N = 7 with no redesign.

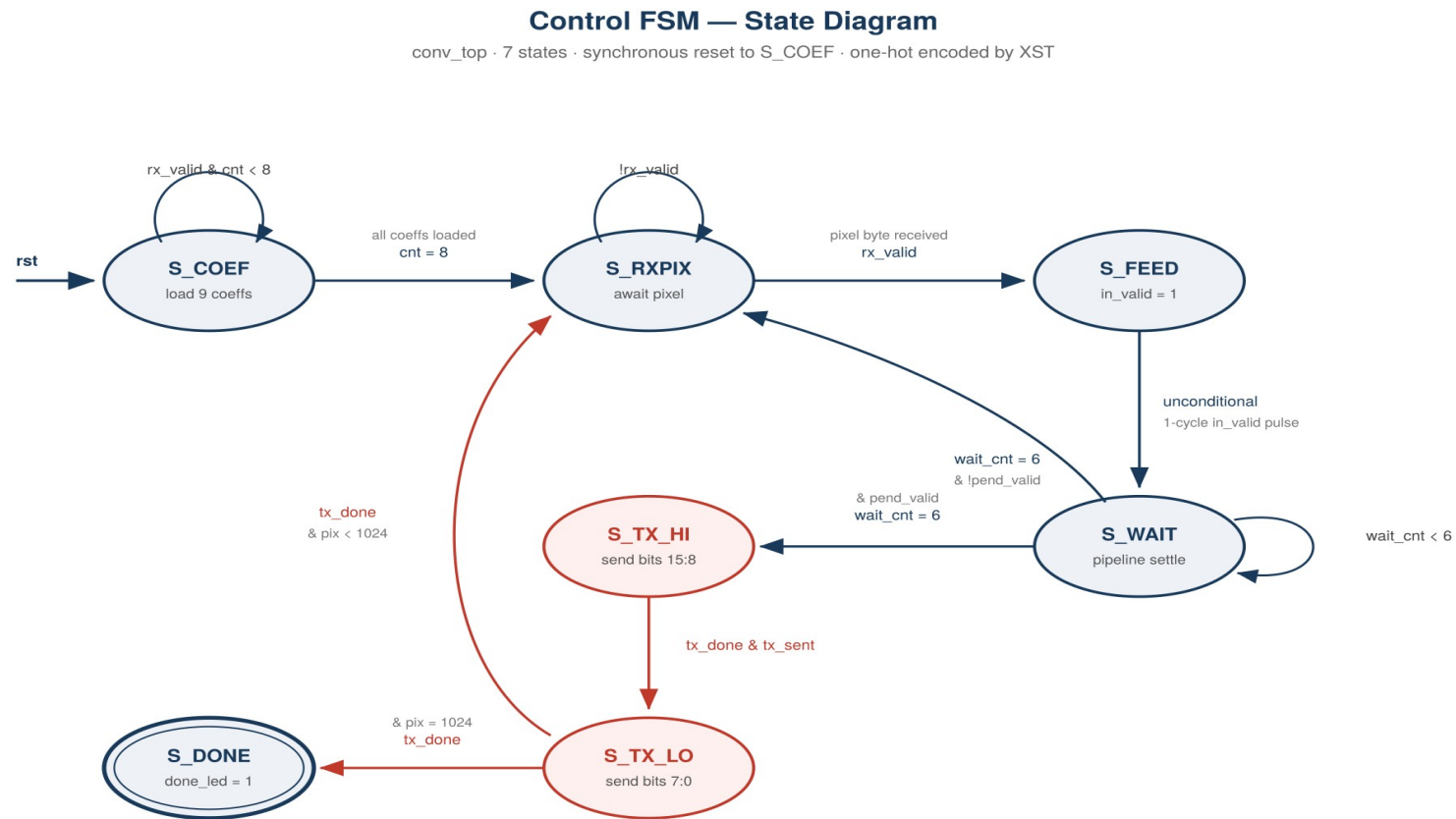
Saturation is mandatory

The accumulator range exceeds the 16-bit output range by roughly 9×, so results are clamped to -32,768 .. +32,767 rather than allowed to wrap.

20 of the 72 verification cases drive saturation deliberately, using all-+127 and all--128 kernels against maximum-intensity images.

Saturation logic that never fires in simulation is untested logic.

Control and host interface



Notes

Request-response protocol: the host sends 9 kernel bytes, then one pixel at a time; the board replies with 2 bytes (big-endian int16) whenever a pixel completes a window (row ≥ 2 and col ≥ 2). No flow-control lines are needed and no bytes can be dropped.

tx_sent flag: uart_tx clears tx_busy on the same edge it pulses tx_done, so ltx_busy cannot test whether a byte is still owed — doing so re-arms the transmitter on the completion cycle and resends indefinitely. tx_sent records that the byte was handed over.

Verified at three levels

Core

72 cases

6 images × 6 kernels × ReLU on/off,
900 pixels each

72 / 72 bit-exact

System

6 cases

Full UART protocol including
coefficient load and readback

900 / 900 each

Host

8 cases

Protocol sequencing with no
hardware attached (dry run)

All pass

Two defects found — both real synchronous-design hazards

Window-valid misalignment

Window registers and position counters update on the same edge, so the valid flag led the data by one cycle and every output shifted by one position. Fixed with a registered win_valid_q stage.

UART transmit handshake race

uart_tx clears tx_busy on the same edge it pulses tx_done, so testing !tx_busy re-armed the transmitter and it resent the same byte forever. Fixed with an explicit tx_sent flag.

We measured instead of assuming

FOM = Throughput ÷ [Power × (LUTs + 50×DSPs + 100×BRAMs)]

Nine DSP48E blocks contribute 450 units — 60% of the resource denominator. The natural hypothesis: move the multipliers into LUT logic and the FoM improves. So we built both.

	Build A — DSP48E	Build B — LUT multipliers
Slice LUTs	317	1,318
DSP48E	9	0
Block RAM	0	0
Resource cost	767	1,318
F-max (post-route)	167.0 MHz	132.8 MHz
Figure of Merit	2.398×10 ⁻³	1.395×10 ⁻³

The hypothesis was wrong

Each 9×8 signed multiplier costs about 113 LUTs in Virtex-5 fabric.

Removing the DSPs saved 450 penalty units but added 1,001 LUTs — a net 551 units worse, plus a 24% loss of maximum frequency.

Build A ships, with a 72% higher Figure of Merit. Reasoning from the formula alone would have selected the worse design.

Earning the one-output-per-cycle bonus

The organizer's bar: out_valid must never drop for 2+ cycles during processing — not even at row boundaries.

First attempt: padding pixels

Injected literal zero pixels between rows to zero-pad the math. Correct values — but each injected pixel cost one real cycle: 2-cycle stall at every row boundary, 31 violations measured directly.

Fix: mask, don't inject

Top padding comes free from line-buffer reset timing. Left padding is applied by masking the window's oldest columns to zero — no pixel ever costs a cycle. Every real pixel produces one output.

Proven, not asserted

A dedicated testbench feeds the core 1,024 real pixels continuously — no padding pixels of any kind — and a cycle-by-cycle monitor tracks every gap in out_valid.

Result: 1024 / 1024 bit-exact · max gap = 0 cycles · violations ≥ 2 cycles = 0

Trade-off: +118 LUTs for the masking logic and output FIFO, throughput 0.8755 → 1.0. Net FoM change: -1.0% — reported honestly, not oversold.

Zero block RAM, by construction

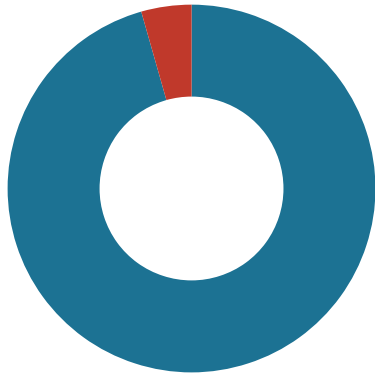
The line buffers

Two 256-bit flat shift registers, which XST infers as sixteen 30-bit SRL primitives occupying 17 LUTs in total — confirmed in the map report.

Why it matters

One BRAM would have cost at least 100 penalty units — more than a third of the entire LUT budget of the submitted design — to store 512 bits.

Power breakdown



■ Static leakage (455 mW) ■ Dynamic (21 mW)

Only 21 mW of the 476 mW total is dynamic. The remaining 455 mW is static leakage inherent to the 65 nm Virtex-5 process and independent of the design — it is the same figure our earlier work measured on this board.

Against dynamic power alone the FoM would be 5.52×10^{-2} , some 23× higher. We report total power as the official figure, as the conservative reading of the specification.

Measured results

Parameter	VALID (primary)	SAME (zero-gap bonus)	Units
Input / output precision	8 / 16, saturating	8 / 16, saturating	bits
Architecture	Line-buffer sliding window	+ column masking, output FIFO	—
Multipliers	9 (DSP48E)	9 (DSP48E)	—
Latency, first output	70	70	cycles
Frame cycles (32×32)	1,028	1,024	cycles
Throughput, sustained	0.8755	1.0000	out px/cycle
FPGA utilisation	317 LUT, 275 FF	435 LUT, 325 FF	—
Block RAM	0	0	—
Maximum frequency	167.0	165.7	MHz
Timing status	All met, score 0	All met, score 0	—
Power estimate	476 (21 dyn / 455 static)	476 (21 dyn / 455 static)	mW
Verification	78/78 bit-exact	1024/1024, 0-cycle max gap	—
FOM	2.398 × 10 ⁻³	2.374 × 10 ⁻³	—

What we would claim

1 Deterministic and lean

One output pixel per cycle, zero BRAM, 317 LUTs, 67% frequency headroom.

2 Verified, not asserted

78 cases bit-exact against a golden model, including deliberate saturation coverage.

3 Measured, not assumed

Two full builds placed and routed to settle the DSP-versus-LUT question empirically.

4 Honest about limits

Total power reported despite 96% of it being process leakage we cannot influence.

Remaining: board demonstration and an edge-detection demo — both optional bonus items. The bitstream is built.